AWS S3

You can enable the AWS S3 wrapper right from the Supabase dashboard.

Open wrapper in dashboard

AWS S3 is an object storage service offering industry-leading scalability, data availability, security, and performance. It is read-only and supports below file formats:

The S3 Wrapper allows you to read data of below formats from S3 within your Postgres database.

1. CSV - with or without header line
2. JSON Lines
3. Parquet

The S3 Wrapper also supports below compression algorithms:

1. gzip
2. bzip2
3. xz
4. zlib

Note for CSV and JSONL files: currently all columns in S3 files must be defined in the foreign table and their types must be `text` type.

Note for Parquet files: the whole Parquet file will be loaded into local memory if it is compressed, so keep the file size as small as possible.

Preparation

Before you can query S3, you need to enable the Wrappers extension and store your credentials in Postgres.

Enable Wrappers

Make sure the `wrappers` extension is installed on your database:

```
create extension if not exists wrappers with schema extensions;
```

Enable the S3 Wrapper

Enable the `s3_wrapper` FDW:

```
create foreign data wrapper s3_wrapper
  handler s3_fdw_handler
  validator s3_fdw_validator;
```

Store your credentials (optional)

By default, Postgres stores FDW credentials inside `pg_catalog.pg_foreign_server` in plain text. Anyone with access to this table will be able to view these credentials. Wrappers is designed to work with Vault, which provides an additional level of security for storing credentials. We recommend using Vault to store your credentials.

```
-- Save your AWS credentials in Vault and retrieve the created `s3_access_key
```

```
select vault.create_secret(
  '<access key id>',
  's3_access_key_id',
  'AWS access key for Wrappers'
);
select vault.create_secret(
  '<secret access key>'
  's3_secret_access_key',
  'AWS secret access key for Wrappers'
```

Database

Connecting to S3

We need to provide Postgres with the credentials to connect to S3, and any additional options. We can do this using the `create server` command:

With Vault | Without Vault

```
create server s3_server
  foreign data wrapper s3_wrapper
  options (
    vault_access_key_id '<your s3_access_key_id from above>',
    vault_secret_access_key '<your s3_secret_access_key from above>',
    aws_region 'us-east-1'
  );
```

The full list of options are below:

- `aws_access_key_id` (required) - Your access key
- `aws_secret_access_key` (required) - Your secret key
- `aws_region` (required) - The region of your bucket (if providing an endpoint URL with a region in it, make sure that they are the same)
- `endpoint_url` (optional) - An optional URL to allow connection to S3-compliant providers (i.e. Wasabi, Cloudflare R2, Backblaze B2, DigitalOcean Spaces)

`path_style_url` (optional) - Whether to use path-style URL access. This is required by some S3-compliant providers. `true` or `false`, default is `false`.

Required S3 permissions

Below S3 permissions are needed:

s3:GetObject

s3:GetObjectAttributes

If the bucket is versioned, we also need:

s3:GetObjectVersion

s3:GetObjectVersionAttributes

Connecting to S3-compliant Providers - Supabase Storage

```
create server s3_server
  foreign data wrapper s3_wrapper
  options (
    aws_access_key_id '<supabase_storage_access_key>',
    aws_secret_access_key '<supabase_storage_secret_key>',
    aws_region 'eu-central-1',
    endpoint_url 'https://<project_ref>.supabase.co/storage/v1/s3',
    path_style_url 'true'
  );
```

Connecting to S3-compliant Providers - Wasabi

```
create server s3_server
  foreign data wrapper s3_wrapper
  options (
    aws_access_key_id '<your_wasabi_access_key>',
    aws_secret_access_key '<your_wasabi_secret_key>',
    aws_region 'eu-central-1',
```

```
      endpoint_url 'https://s3.eu-central-1.wasabisys.com'
    );
```

Create a schema

We recommend creating a schema to hold all the foreign tables:

```
create schema if not exists s3;
```

Options

The following options are available when creating S3 foreign tables:

- `uri` - S3 URI, required. For example, `s3://bucket/s3_table.csv`
- `format` - File format, required. `csv`, `jsonl`, or `parquet`
- `has_header` - If the CSV file has header, optional. `true` or `false`, default is `false`
- `compress` - Compression algorithm, optional. One of `gzip`, `bzip2`, `xz`, `zlib`, default is no compression

Entities

CSV Files

This is an object representing CSV files in S3.

Ref: AWS S3 docs

Operations

Object	Select	Insert	Update	Delete	Truncate
CSV	✅	❌	❌	❌	❌

Usage

```
create foreign table s3.table_csv (
  name text,
  sex text,
  age text,
  height text,
  weight text
)
  server s3_server
  options (
    uri 's3://bucket/s3_table.csv',
    format 'csv',
    has_header 'true'
  );
```

Notes

- All columns must be defined in the foreign table
- All column types must be `text`
- Optional header support via `has_header` option
- Supports compression (gzip, bzip2, xz, zlib)

JSON Lines Files

This is an object representing JSONL files in S3.

Ref: JSONL docs

Operations

Object	Select	Insert	Update	Delete	Truncate
JSONL	✅	❌	❌	❌	❌

Usage

```
create foreign table s3.table_jsonl (
  name text,
  sex text,
  age text,
  height text,
  weight text
)
  server s3_server
  options (
    uri 's3://bucket/s3_table.jsonl',
    format 'jsonl'
  );
```

Notes

All columns must be defined in the foreign table

All column types must be `text`

Each line must be a valid JSON object

Supports compression (gzip, bzip2, xz, zlib)

Parquet Files

This is an object representing Parquet files in S3.

Ref: Parquet docs

Operations

Object	Select	Insert	Update	Delete	Truncate
Parquet	✅	❌	❌	❌	❌

Usage

```
create foreign table s3.table_parquet (
  id integer,
  bool_col boolean,
  bigint_col bigint,
  float_col real,
  date_string_col text,
  timestamp_col timestamp
)
  server s3_server
  options (
    uri 's3://bucket/s3_table.parquet',
    format 'parquet'
  );
```

Notes

- Not all columns need to be defined in foreign table
- Column names must match between Parquet file and foreign table
- Supports various Postgres data types (see Data Types section)
- Compressed files are loaded entirely into memory
- Supports compression (gzip, bzip2, xz, zlib)

Query Pushdown Support

This FDW doesn't support query pushdown.

Supported Data Types For Parquet File

The S3 Wrapper uses Parquet file data types from arrow_array::types, below are their mappings to Postgres data types.

Postgres Type	Parquet Type
boolean	BooleanType
char	Int8Type
smallint	Int16Type
real	Float32Type
integer	Int32Type
double precision	Float64Type
bigint	Int64Type
numeric	Float64Type
text	ByteArrayType
date	Date64Type
timestamp	TimestampNanosecondType
timestamptz	TimestampNanosecondType

Limitations

This section describes important limitations and considerations when using this FDW:

- Large result sets experience slower performance due to full data transfer requirement
- Compressed files are loaded entirely into memory and have additional processing overhead
- CSV and JSONL columns must be defined as `text` type only
- All columns must be defined in foreign tables for CSV and JSONL
- Column names must match exactly for Parquet files
- No support for S3 Select or other S3-side filtering

Materialized views using these foreign tables may fail during logical backups

Examples

Basic Example

This will create some "foreign table" inside your Postgres database can read data from S3:

```
-- CSV file, no compression
create foreign table s3.table_csv (
  name text,
  sex text,
  age text,
  height text,
  weight text
)
  server s3_server
  options (
    uri 's3://bucket/s3_table.csv',
    format 'csv',
    has_header 'true'
  );

-- JSON line file, no compression
create foreign table s3.table_jsonl (
  name text,
  sex text,
  age text,
  height text,
  weight text
)
  server s3_server
  options (
    uri 's3://bucket/s3_table.jsonl',
    format 'jsonl'
  );

-- GZIP compressed CSV file
create foreign table s3.table_csv_gzip (
  name text,
  sex text,
  age text,
```

```
    height text,
    weight text
  )
    server s3_server
    options (
      uri 's3://bucket/s3_table.csv.gz',
      format 'csv',
      has_header 'true',
      compress 'gzip'
    );

  -- Parquet file, no compression
  create foreign table s3.table_parquet (
    id integer,
    bool_col boolean,
    bigint_col bigint,
    float_col real,
    date_string_col text,
    timestamp_col timestamp
  )
    server s3_server
    options (
      uri 's3://bucket/s3_table.parquet',
      format 'parquet'
    );

  -- GZIP compressed Parquet file
  create foreign table s3.table_parquet_gz (
    id integer,
    bool_col boolean,
    bigint_col bigint,
    float_col real,
    date_string_col text,
    timestamp_col timestamp
  )
    server s3_server
    options (
      uri 's3://bucket/s3_table.parquet.gz',
      format 'parquet',
      compress 'gzip'
    );
```

Read from Supabase Storage

This example will read a CSV file stored on Supabase Storage. The access information can be found on Supabase Storage settings page.

```
create server s3_server
  foreign data wrapper s3_wrapper
  options (
    aws_access_key_id '<access key ID>',
    aws_secret_access_key '<secret access key>',
    aws_region '<region>',
    endpoint_url 'https://<project_ref>.supabase.co/storage/v1/s3',
    path_style_url 'true'
  );

create foreign table s3.supabase_table_csv (
  id text,
  name text
)
  server s3_server
  options (
    uri 's3://mybucket/myfile.csv',
    format 'csv',
    has_header 'true'
  );
```

Edit this page on GitHub

Need some help? Contact support

Latest product updates? See Changelog

Something's not right? Check system status

© Supabase Inc

—

Contributing

Author Styleguide

Open Source

SupaSquad